

Rational Matrix Policy Optimization for Transformer Language Models

Anonymous authors

Paper under double-blind review

Abstract

Transformer nonlinear sublayers contain large input and output matrices around a coordinatewise activation, yet standard optimizers usually update these matrices without using their different functional roles. We study Rational Latent Basis (RLB), a grouped rational activation sublayer that exposes this structure, and MatrixPolicy, an optimizer that uses the exposed matrix roles and rational-curve statistics during language-model pretraining. The main RLB variant uses one global P5/Q4 rational curve per group, initialized from SiLU and trained with no active local atoms. MatrixPolicy combines three role-aware operations: a centered group-gradient policy, a depth-dependent matrix update with a short early Muon branch, and a bounded positive rescaling of each input/output matrix pair. On the completed 100M-token and 300M-token suites across five corpora for a fixed 12-layer, width-768 model, MatrixPolicy reaches lower final validation loss than every matched finite, non-diverged optimizer control; controls that produce non-finite losses are reported as divergent outcomes rather than finite comparators. Token-to-target readouts show that the advantage often appears before the fixed training budget is exhausted.

1 Introduction

Transformer language models rely on nonlinear sublayers that expand the residual stream, apply a coordinatewise activation, and project back to the model width (Vaswani et al., 2017). Modern activation choices such as GELU, SiLU, and SwiGLU improve the scalar nonlinearity or gating pattern (Hendrycks & Gimpel, 2016; Ramachandran et al., 2017; Shazeer, 2020), but the optimizer usually sees the surrounding matrices as ordinary parameter tensors. This loses a useful distinction: the input matrix chooses the coordinates in which the nonlinearity operates, the activation controls local response and saturation, and the output matrix recombines the resulting features.

This paper asks whether exposing that structure can make optimization more effective. We introduce Rational Latent Basis (RLB), a Transformer nonlinear sublayer that places trainable grouped rational curves between an input-domain matrix and an output-recombination matrix. The rational curves are not used as a standalone trick; they define a geometry that the optimizer can observe. Each group has a normalized coordinate system, a trainable scalar response, derivative and output-gain statistics, and a limited positive rescaling relation between the two adjacent matrices.

MatrixPolicy is the optimizer built for this parameterization. It uses live relative-pressure statistics to compare the input and output matrix roles, uses rational derivative/output gains to scale group updates, applies a short early Muon branch to shape the latent directions, and keeps the two matrix representatives balanced by a bounded positive rescaling. The resulting method is intentionally conservative: rational coefficients and all non-RLB backbone parameters remain on AdamW, while MatrixPolicy acts only on the RLB input and output matrices.

The completed evidence in this draft covers language-model runs on five corpora at 100M and 300M training-token budgets for a fixed 12-layer, width-768 model. These runs compare

MatrixPolicy against matched global-rational RLB controls using AdamW, Lion, Muon, SOAP-style, CAME-style, ScheduleFree-style, and ADeMaMix-style optimizers, as well as SiLU controls under the same protocol. Non-finite or early-stopped ADeMaMix-style RLB rows are reported as divergent outcomes rather than finite-loss competitors. The current results show a consistent final-loss gap in favor of MatrixPolicy among finite controls and a token-to-target advantage at several validation-loss thresholds. Larger scaling experiments will be needed before claiming behavior beyond this regime.

2 Related Work

Transformer nonlinear sublayers. The Transformer positionwise nonlinear sublayer was introduced alongside self-attention (Vaswani et al., 2017). Subsequent activation work showed that smooth and gated nonlinearities can improve optimization and quality, including GELU (Hendrycks & Gimpel, 2016), Swish/SiLU (Ramachandran et al., 2017), and GLU variants such as SwiGLU (Shazeer, 2020). RLB follows this line in replacing a fixed scalar activation, but differs by making the activation trainable and grouped while keeping a single input projection and output projection around it. The goal is to change the function class while exposing matrix roles that the optimizer can use.

Pretraining optimizers. Adam and AdamW remain standard choices for Transformer pretraining (Kingma & Ba, 2015; Loshchilov & Hutter, 2019). Recent work explores alternative update rules, parameter factorizations, and curvature or matrix-aware preconditioners for language models, including Lion (Chen et al., 2023), Muon (Liu et al., 2025), Sophia (Liu et al., 2024), GaLore (Zhao et al., 2024), SOAP (Vyas et al., 2025), Adam-mini (Zhang et al., 2025), ADeMaMix (Pagliardini et al., 2025), CAME (Luo et al., 2023), cautious optimizers (Liang et al., 2026), and recent pretraining-optimizer surveys (Wen et al., 2026). MatrixPolicy is closest in spirit to matrix-aware optimization, but it does not apply one generic matrix rule to every two-dimensional tensor. It uses the local RLB structure to decide which matrices receive role-specific scaling, which branch is active early, and which group statistics control the update.

Pretraining corpora. The experiments use web-scale or web-derived corpora that are common in language model pretraining studies, including DCLM/DataComp-LM (Li et al., 2024), FineWeb and FineWeb-Edu (Penedo et al., 2024), Dolma (Soldaini et al., 2024), and C4 from the T5 work (Raffel et al., 2020). We use these corpora to test whether the optimizer effect is tied to one data source or persists across several text distributions.

3 Method

RLB and MatrixPolicy are designed as a coupled activation-optimizer pair. RLB separates a nonlinear sublayer into an input-domain matrix, grouped rational curves, and an output-recombination matrix. MatrixPolicy uses this separation to assign different update rules to the two matrices around each rational curve, while the remaining model parameters and the rational coefficients are updated by AdamW (Kingma & Ba, 2015; Loshchilov & Hutter, 2019). The method does not claim that a trainable rational curve alone is sufficient; the experiments use matched RLB controls with the same global-rational activation and different optimizers to isolate the optimizer effect.

3.1 Global-Rational RLB

For Transformer layer $l \in \{1, \dots, L\}$, let $x_l \in \mathbb{R}^d$ denote the input to the nonlinear sublayer. RLB first projects this input to a latent width $H = Gm$ and partitions the latent vector into G groups of width m :

$$z_l = A_l x_l, \quad z_l = \text{concat}_{g=1}^G z_{l,g}, \quad z_{l,g} \in \mathbb{R}^m, \quad (1)$$

where $A_l \in \mathbb{R}^{H \times d}$ is the input-domain matrix. Each group is normalized by its root-mean-square radius,

$$r_{l,g} = \sqrt{m^{-1} \|z_{l,g}\|_2^2 + \epsilon_{\text{rms}}}, \quad u_{l,g} = z_{l,g}/r_{l,g}, \quad (2)$$

with numerical floor $\epsilon_{\text{rms}} > 0$. A scalar rational curve is applied coordinatewise in the normalized coordinates and then multiplied by the same radius:

$$h_{l,g} = r_{l,g} R_{l,g}(u_{l,g}), \quad h_l = \text{concat}_{g=1}^G h_{l,g}, \quad y_l = B_l h_l, \quad (3)$$

where $B_l \in \mathbb{R}^{d \times H}$ is the output-recombination matrix.

The curve $R_{l,g}$ is a trainable global rational function,

$$R_{l,g}(u) = \frac{P_{l,g}(u)}{Q_{l,g}(u)}, \quad P_{l,g}(u) = \sum_{i=0}^5 a_{l,g,i} u^i, \quad (4)$$

with denominator

$$Q_{l,g}(u) = 1 + |b_{l,g,1}| |u| + |b_{l,g,2}| u^2 + |b_{l,g,3}| |u|^3 + |b_{l,g,4}| u^4. \quad (5)$$

The numerator and denominator coefficients are initialized by fitting SiLU on a fixed probe interval and are trained thereafter. The denominator form keeps the real response pole-free because all nonconstant terms are nonnegative on the real line, while still allowing the learned curve to change slope, curvature, and saturation across layers and groups. In the main variant there are no active local Cauchy atoms, no trainable atom centers, and no atom coefficient parameters; the nonlinear response is the grouped P5/Q4 curve itself.

The normalization in equation 2 gives the optimizer two views of each group. The normalized coordinate $u_{l,g}$ determines where the rational curve is evaluated, while the radius $r_{l,g}$ carries group scale back to the residual stream. In the idealized case $\epsilon_{\text{rms}} = 0$ with nonzero group radii, this construction admits the positive rescaling

$$f_l(x; A_l, B_l, R_l) = f_l(x; D_l(a) A_l, B_l D_l(a)^{-1}, R_l), \quad (6)$$

where $R_l = \{R_{l,g}\}_{g=1}^G$ and $D_l(a) = \text{diag}(a_1 I_m, \dots, a_G I_m)$ for $a \in \mathbb{R}_{>0}^G$. The RMS floor makes this identity approximate near degenerate groups, so MatrixPolicy uses it only as the mathematical basis for a small bounded rescaling of the surrounding matrices.

3.2 MatrixPolicy Signals

MatrixPolicy treats A_l and B_l as different matrix roles around the same rational curve. It therefore tracks role-specific pressure and curve-gain statistics for each layer and group. For any tensor V , define the floored RMS $\text{rms}_{\epsilon}(V) = \sqrt{\text{mean}(V^2) + \epsilon}$. All log-domain ratios below use a small positive floor before taking logarithms. Let $A_{l,g} \in \mathbb{R}^{m \times d}$ be the row block of A_l , and let $B_{l,g} \in \mathbb{R}^{d \times m}$ be the column block of B_l . The relative matrix-gradient pressures are

$$p_{l,g}^{\text{in}} = \frac{\text{rms}_{\epsilon_p}(\nabla A_{l,g})}{\max\{\text{rms}_{\epsilon_p}(A_{l,g}), \epsilon_p\}}, \quad p_{l,g}^{\text{out}} = \frac{\text{rms}_{\epsilon_p}(\nabla B_{l,g})}{\max\{\text{rms}_{\epsilon_p}(B_{l,g}), \epsilon_p\}}. \quad (7)$$

The rational-activity statistic for the global-rational curve is

$$p_{l,g}^R = \left[\frac{1}{2} (\text{mean}((\nabla a_{l,g,\cdot})^2) + \text{mean}((\nabla b_{l,g,\cdot})^2)) + \epsilon_p \right]^{1/2}. \quad (8)$$

MatrixPolicy maintains exponential moving averages $q_{l,g}^{\text{in}}$, $q_{l,g}^{\text{out}}$, and $q_{l,g}^R$ of these quantities with decay 0.95, and forms

$$\pi_{l,g} = \log q_{l,g}^{\text{in}} - \log q_{l,g}^{\text{out}}, \quad \alpha_{l,g} = \log q_{l,g}^R - \frac{1}{2} (\log q_{l,g}^{\text{in}} + \log q_{l,g}^{\text{out}}). \quad (9)$$

Here $\pi_{l,g}$ measures relative update pressure between the input and output matrices, while $\alpha_{l,g}$ measures whether the rational coefficients are receiving unusually large gradients relative to the adjacent matrices.

The activation also provides curve-gain signals. From a recent sample of normalized coordinates $u_{l,g}$ cached by the RLB forward path, MatrixPolicy uses

$$d_{l,g}^{\text{live}} = \text{rms}(R'_{l,g}(u_{l,g})), \quad o_{l,g}^{\text{live}} = \text{rms}(R_{l,g}(u_{l,g})). \quad (10)$$

These statistics are not full block Jacobians. They are cheap proxies: derivative gain reflects the sensitivity of the input-domain role, and output gain reflects the scale seen by the recombination role.

3.3 MatrixPolicy Update

Let t be the optimizer step, T the planned number of training steps, and $\xi_t = t/T$. A smooth gate

$$\phi_t = \text{smoothstep}(0.02, 0.30, \xi_t) \quad (11)$$

turns on the group-gradient policy. For positive group values v_j , write $\text{geomean}_j v_j = \exp(G^{-1} \sum_{j=1}^G \log v_j)$. The multiplier for group g and role $r \in \{\text{in}, \text{out}\}$ is

$$c_{l,g,r} = c_{l,g,r}^{\text{gain}} c_{l,g,r}^{\text{pressure}} c_{l,g}^{\text{activity}}. \quad (12)$$

For the input role, the gain term uses $d_{l,g}^{\text{live}}$; for the output role, it uses $o_{l,g}^{\text{live}}$:

$$c_{l,g,\text{in}}^{\text{gain}} = \left(\frac{\text{geomean}_j d_{l,j}^{\text{live}}}{d_{l,g}^{\text{live}}} \right)^{0.20\phi_t}, \quad c_{l,g,\text{out}}^{\text{gain}} = \left(\frac{\text{geomean}_j o_{l,j}^{\text{live}}}{o_{l,g}^{\text{live}}} \right)^{0.20\phi_t}. \quad (13)$$

With $\tilde{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.50, 1.50)$, pressure shifts update budget from the higher-pressure role toward the lower-pressure role:

$$c_{l,g,\text{in}}^{\text{pressure}} = \exp(-0.10\phi_t \tilde{\pi}_{l,g}), \quad c_{l,g,\text{out}}^{\text{pressure}} = \exp(+0.10\phi_t \tilde{\pi}_{l,g}). \quad (14)$$

Rational activity damps both matrix roles when the curve coefficients are moving more strongly than the adjacent matrices:

$$c_{l,g}^{\text{activity}} = \exp \left[-0.20\phi_t \text{ReLU} \left(\frac{\alpha_{l,g} - 0.05}{0.45} \right) \right]. \quad (15)$$

The final multiplier is recentered and clipped,

$$c_{l,g,r} \leftarrow \frac{c_{l,g,r}}{\text{geomean}_j c_{l,j,r}}, \quad c_{l,g,r} \leftarrow \text{clip}(c_{l,g,r}, 0.75, 1.35), \quad (16)$$

so the policy redistributes update scale across groups without changing the mean log learning-rate multiplier. All completed MatrixPolicy rows use this fixed group-policy configuration across datasets and seeds.

After group-gradient scaling, the RLB matrices are updated by a role/depth AdamW branch and a short early Muon branch. Define $\delta_l = (l-1)/(L-1)$ for $L > 1$ and $\delta_l = 1/2$ otherwise. The role factors are

$$\rho_{\text{in}}(l) = \text{clip}(1 - 0.50(\delta_l - 0.5), 0.55, 1.40), \quad \rho_{\text{out}}(l) = \text{clip}(1 + 1.00(\delta_l - 0.5), 0.55, 1.40), \quad (17)$$

and the AdamW matrix multiplier for role r is

$$s_{l,r}^{\text{Adam}} = \text{clip}(3.0 \max\{0.10, 1 + 1.20(\rho_r(l) - 1)\}, 0.40, 4.0). \quad (18)$$

The Muon branch is concentrated early in training and is reduced when pressure imbalance or rational activity is high. Define

$$\Psi_{l,t} = \frac{1}{G} \sum_{g=1}^G |\text{clip}(\pi_{l,g}, -1.50, 1.50)|, \quad \Omega_{l,t} = \frac{1}{G} \sum_{g=1}^G \text{ReLU} \left(\frac{\alpha_{l,g} - 0.05}{0.45} \right), \quad (19)$$

$$\psi_{l,t} = \text{clip}(\exp[-0.30\Psi_{l,t} - 0.65\Omega_{l,t}], 0.10, 1.15). \quad (20)$$

The branch fraction is

$$\mu_{l,r,t} = \text{clip}(0.75 \text{ smoothstep}(0.02, 0.12, \xi_t)[1 - \text{smoothstep}(0.20, 0.36, \xi_t)]\rho_r(l)\psi_{l,t}, 0, 0.75). \quad (21)$$

For $M_{l,\text{in}} = A_l$ and $M_{l,\text{out}} = B_l$, MatrixPolicy composes a stateful AdamW matrix update with the Muon branch:

$$\begin{aligned} \widetilde{M}_{l,r}^{t+1} &= \text{AdamW}_{\eta_t s_{l,r}^{\text{Adam}}(1-\mu_{l,r,t})}(M_{l,r}^t; \nabla M_{l,r}^t), \\ M_{l,r}^{t+1} &= \text{Muon}_{\eta_t \mu_{l,r,t}}(\widetilde{M}_{l,r}^{t+1}; \nabla M_{l,r}^t), \end{aligned} \quad (22)$$

where η_t is the base learning rate and Muon denotes the `torch.optim.Muon` update on two-dimensional RLB matrix parameters: momentum 0.95, five Newton–Schulz orthogonalization steps, and the match-RMS learning rate adjustment of the Muon optimizer (Liu et al., 2025). In the implementation, the AdamW child optimizer is stepped first with learning-rate factor $s_{l,r}^{\text{Adam}}(1 - \mu_{l,r,t})$, and the Muon child optimizer is then stepped with learning-rate factor $\mu_{l,r,t}$. When all Muon fractions have decayed to zero, this child step is inactive.

The last MatrixPolicy operation is a bounded positive rescaling of the two matrix blocks around each rational curve. It is evaluated every five optimizer steps; on other steps this operation is skipped. On a fixed probe grid, define

$$d_{l,g}^{\text{probe}} = \text{rms}(R'_{l,g}(u)), \quad o_{l,g}^{\text{probe}} = \text{rms}(R_{l,g}(u)), \quad (23)$$

and the current matrix log ratio

$$\gamma_{l,g} = \log \text{rms}(A_{l,g}) - \log \text{rms}(B_{l,g}). \quad (24)$$

The target ratio combines a curve-gain term with a bounded pressure term,

$$\tau_{l,g} = \frac{1}{2} \left(\log d_{l,g}^{\text{probe}} - \log o_{l,g}^{\text{probe}} \right) + 0.25 \bar{\pi}_{l,g}, \quad \bar{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.25, 1.25). \quad (25)$$

Rational activity modulates the rescaling strength through

$$a_{l,g}^{\text{rs}} = \exp(\text{clip}(0.10 \bar{\alpha}_{l,g}, \log 0.75, \log 1.25)), \quad \bar{\alpha}_{l,g} = \text{clip}(\alpha_{l,g}, -1.25, 1.25). \quad (26)$$

With schedule

$$s_{l,t} = 0.50 \text{ smoothstep}(0.03, 0.35, \xi_t) \text{ clip}(1 + 0.15(\delta_l - 0.5), 0.75, 1.25), \quad (27)$$

On active rescaling steps, the log move is

$$\ell_{l,g,t} = \text{clip}\left(\frac{1}{2}[\tau_{l,g} - \gamma_{l,g}]s_{l,t}a_{l,g}^{\text{rs}}, -0.030, 0.030\right). \quad (28)$$

On active rescaling steps, the matrix blocks are updated as

$$A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}, \quad B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}. \quad (29)$$

This operation follows the positive rescaling structure in equation 6, but it is deliberately clipped because the RMS floor and optimizer state make the implemented update only approximately representation-preserving.

Algorithm 1: MatrixPolicy step for global-rational RLB.

1. Update matrix-pressure and rational-activity moving averages from the current gradients.
2. Read the most recent cached curve-gain statistics from the RLB forward path.
3. Step non-matrix parameters, including rational coefficients, with AdamW.
4. In the matrix child optimizer, apply the centered group-gradient multiplier in equation 16 to RLB matrix gradients.
5. Step RLB matrices with the AdamW child step and the active Muon child step in equation 22.
6. Every five optimizer steps, apply the bounded positive rescaling in equation 29; otherwise skip this operation.

Table 1: Final validation loss for the completed M0 runs. Values are mean over three seeds; lower is better. MP denotes MatrixPolicy. “Best non-MP” is the strongest finite non-MatrixPolicy aggregate method in that dataset and budget.

Dataset	E1 MP	E1 best	E1 gap	E2 MP	E2 best	E2 gap
DCLM	4.2538	4.2946	0.0408	3.9518	3.9887	0.0369
FineWeb-Edu	4.0873	4.1361	0.0488	3.7015	3.7373	0.0358
FineWeb	4.3162	4.3626	0.0464	3.9623	3.9960	0.0337
Dolma-sample	4.3253	4.3622	0.0369	3.8062	3.8412	0.0351
C4	4.2837	4.3271	0.0434	3.8777	3.9132	0.0355

Table 2: Representative E2 token-to-target readouts. MP denotes MatrixPolicy. Tokens are millions of training tokens needed to first reach the target validation loss; lower is better.

Dataset	Target loss	MatrixPolicy	Best non-MP	Tokens saved
DCLM	4.05	201.0M	220.1M	8.7%
FineWeb-Edu	3.80	209.7M	223.4M	6.1%
FineWeb	4.05	213.0M	228.8M	6.9%
Dolma-sample	3.85	242.5M	280.2M	13.5%
C4	4.00	195.0M	204.8M	4.8%

4 Experiments

Setup. The completed runs use a 12-layer, width-768 causal Transformer, denoted M0 in the result artifacts, with sequence length 256 and 32768 global tokens per optimizer step. E1 trains for 3050 steps (about 100M tokens), and E2 trains for 9150 steps (about 300M tokens). Each cell uses three seeds on DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4. MatrixPolicy is compared against matched RLB controls using AdamW, Lion, Muon, SOAP-style, CAME-style, ScheduleFree-style, and ADeMaMix-style optimizers, and against SiLU controls under the same model and token budgets. Non-finite or early-stopped ADeMaMix-style RLB rows are reported as divergent outcomes, not as finite final-loss comparators.

Final validation loss. Table 1 summarizes the completed M0 results. In E1, MatrixPolicy has the lowest mean final validation loss on all five datasets; the best finite non-MatrixPolicy method is RLB+Lion in each cell. In E2, MatrixPolicy is again the lowest-loss finite method on all five datasets; the best finite non-MatrixPolicy method is RLB+Lion except on FineWeb-Edu, where RLB+Muon is the strongest aggregate comparator. The E2 gaps are smaller than in E1, but they persist after three times as many tokens.

Token-to-target behavior. The fixed-budget finite-loss comparison does not show whether a method reaches useful loss levels earlier. Table 2 reports a representative hard threshold from each completed E2 dataset where MatrixPolicy and the strongest non-MatrixPolicy comparator both reach the target on all three seeds. The readout uses the native 50-step validation cadence, so token differences are quantized in intervals of 1.64M tokens. MatrixPolicy reaches these thresholds with fewer tokens in every listed dataset.

Interpretation. These results support two narrow conclusions. First, the global-rational activation is not by itself the whole effect: RLB+AdamW is close to SiLU+AdamW, and the strongest finite non-MatrixPolicy RLB optimizer is usually RLB+Lion or RLB+Muon. Second, the role-aware optimizer consistently improves the RLB sublayer under the completed M0 protocol. The evidence does not yet establish scaling behavior beyond 300M tokens or larger model sizes; those larger runs are the next experimental stage.

References

- Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Lu, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic Discovery of Optimization Algorithms. In *Advances in Neural Information Processing Systems*, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Aaron Defazio, Xingyu Yang, Ahmed Khaled, Konstantin Mishchenko, Harsh Mehta, and Ashok Cutkosky. The Road Less Scheduled. In *Advances in Neural Information Processing Systems*, 2024. URL <https://neurips.cc/virtual/2024/poster/96925>.
- Dan Hendrycks and Kevin Gimpel. Gaussian Error Linear Units (GELUs). arXiv preprint arXiv:1606.08415, 2016. URL <https://arxiv.org/abs/1606.08415>.
- Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015. URL <https://mlanthology.org/iclr/2015/kingma2015iclr-adam/>.
- Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, et al. DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0455. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets_and_Benchmarks_Track.html.
- Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious Optimizers: Improving Training with One Line of Code. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=zBPZeRjfgu>.
- Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/06960915ba8674c7a898ec0b472b80ff-Abstract-Conference.html.
- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is Scalable for LLM Training. arXiv preprint arXiv:2502.16982, 2025. URL <https://arxiv.org/abs/2502.16982>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Yang Luo, Xiaozhe Ren, Zangwei Zheng, Zhuo Jiang, Xin Jiang, and Yang You. CAME: Confidence-guided Adaptive Memory Efficient Optimization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pp. 4442–4453, 2023. doi: 10.18653/v1/2023.acl-long.243. URL <https://aclanthology.org/2023.acl-long.243/>.
- Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix Optimizer: Better, Faster, Older. In *International Conference on Learning Representations*, 2025. URL <https://iclr.cc/virtual/2025/poster/28625>.
- Guilherme Penedo, Hynek Kydlicek, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0970. URL https://papers.nips.cc/paper/2024/hash/370df50ccfd8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.

- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL <http://jmlr.org/papers/v21/20-074.html>.
- Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for Activation Functions. arXiv preprint arXiv:1710.05941, 2017. URL <https://arxiv.org/abs/1710.05941>.
- Noam Shazeer. GLU Variants Improve Transformer. arXiv preprint arXiv:2002.05202, 2020. URL <https://arxiv.org/abs/2002.05202>.
- Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, et al. Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. arXiv preprint arXiv:2402.00159, 2024. URL <https://arxiv.org/abs/2402.00159>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 2017. URL https://papers.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
- Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/e988664070e9591f93fdcf605f7dc623-Abstract-Conference.html.
- Kaiyue Wen, David Leo Wright Hall, Tengyu Ma, and Percy Liang. Fantastic Pretraining Optimizers and Where to Find Them. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=2J51qUZ0iG>.
- Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Diederik P. Kingma, Yinyu Ye, Zhi-Quan Luo, and Ruoyu Sun. Adam-mini: Use Fewer Learning Rates To Gain More. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/45ae878717399e6f62d57c65f052cd46-Abstract-Conference.html.
- Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuan-dong Tian. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. In *International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=hYHsrKDiX7>.

A Algebra of the Global-Rational RLB Sublayer

A.1 Pole-free rational response and coefficient derivatives

For one layer and group, write

$$P(u) = \sum_{i=0}^5 a_i u^i, \quad Q(u) = 1 + \sum_{j=1}^4 |b_j| \phi_j(u), \quad \phi(u) = (|u|, u^2, |u|^3, u^4). \quad (30)$$

Since $\phi_j(u) \geq 0$ for all real u , $Q(u) \geq 1$ and $R(u) = P(u)/Q(u)$ has no real pole. Away from the nondifferentiable points of the absolute-value terms, the coordinate derivative is

$$R'(u) = \frac{P'(u)Q(u) - P(u)Q'(u)}{Q(u)^2}, \quad (31)$$

where

$$P'(u) = \sum_{i=1}^5 i a_i u^{i-1}, \quad Q'(u) = |b_1| \text{sign}(u) + 2|b_2|u + 3|b_3|u|u| + 4|b_4|u^3. \quad (32)$$

The coefficient features are similarly explicit:

$$\frac{\partial R(u)}{\partial a_i} = \frac{u^i}{Q(u)}, \quad \frac{\partial R(u)}{\partial b_j} = -\frac{P(u)}{Q(u)^2} \text{sign}(b_j) \phi_j(u), \quad (33)$$

with the usual subgradient convention at zero. Thus the numerator updates change the response through normalized polynomial features, while denominator updates change the response through the same curve multiplied by pole-safe denominator features. This is the mathematical reason the global curve can expose output and derivative gains without introducing separate local-atom parameters.

A.2 Jacobian of RMS-normalized rational groups

For a group vector $z \in \mathbb{R}^m$, define

$$r(z) = \sqrt{m^{-1}\|z\|_2^2 + \epsilon_{\text{rms}}}, \quad u(z) = z/r(z), \quad h(z) = r(z)R(u(z)), \quad (34)$$

where R is applied coordinatewise. The radius derivative is

$$dr = \frac{z^\top dz}{mr} = \frac{u^\top dz}{m}, \quad (35)$$

and the normalized-coordinate derivative is

$$du = \frac{1}{r} \left(I - \frac{uu^\top}{m} \right) dz. \quad (36)$$

Therefore the group Jacobian is

$$J_h(z) := \frac{\partial h}{\partial z} = R(u) \frac{u^\top}{m} + \text{diag}(R'(u)) \left(I - \frac{uu^\top}{m} \right). \quad (37)$$

Equation 37 explains why MatrixPolicy separates output gain and derivative gain. The vector $R(u)$ controls the direct feature scale seen by B_l , while $R'(u)$ appears in the tangent component of the input Jacobian seen by A_l . The RMS-normalization term subtracts the radial component $uu^\top/m$, so the derivative gain is only a proxy for the full group Jacobian, not the full Jacobian itself.

A.3 Positive rescaling and the RMS-floor error

Set $\epsilon_{\text{rms}} = 0$ and take $a > 0$. Then

$$r(az) = ar(z), \quad u(az) = u(z), \quad h(az) = ah(z). \quad (38)$$

Scaling $A_{l,g}$ by a and the matching columns of $B_{l,g}$ by a^{-1} therefore leaves the group contribution unchanged. Applying this independently to all groups gives equation 6.

With $\epsilon_{\text{rms}} > 0$, define r_ϵ and u_ϵ by equation 2. For $\rho^2 = m^{-1}\|z\|_2^2$,

$$u_\epsilon(az) = \kappa(a, z)u_\epsilon(z), \quad \kappa(a, z) = \frac{a\sqrt{\rho^2 + \epsilon_{\text{rms}}}}{\sqrt{a^2\rho^2 + \epsilon_{\text{rms}}}}. \quad (39)$$

Let L_R be a Lipschitz constant of the coordinatewise curve on the segment between $u_\epsilon(z)$ and $\kappa(a, z)u_\epsilon(z)$. After compensating the output matrix by a^{-1} , the group-feature error obeys

$$\|a^{-1}h_\epsilon(az) - h_\epsilon(z)\|_2 \leq \left| \frac{r_\epsilon(az)}{a} - r_\epsilon(z) \right| \|R(u_\epsilon(z))\|_2 \quad (40)$$

$$+ \frac{r_\epsilon(az)}{a} L_R |\kappa(a, z) - 1| \|u_\epsilon(z)\|_2. \quad (41)$$

Both terms vanish when $\epsilon_{\text{rms}} = 0$. For $\rho^2 \gg \epsilon_{\text{rms}}$, $\kappa(a, z) \approx 1$ and the bound is small; near the floor, the approximation can be poor. This is why the rescaling step in equation 28 is bounded rather than treated as an exact function-preserving operation.

B Mathematical Rationale for MatrixPolicy

B.1 Role-specific gradients around an RLB group

For one training example, let $\bar{y}_l = \nabla_{y_l} \mathcal{L}$ be the upstream gradient, and let $J_{l,g} = \partial h_{l,g} / \partial z_{l,g}$ be the group Jacobian in equation 37. The two matrix gradients have different forms:

$$\nabla_{B_{l,g}} \mathcal{L} = \bar{y}_l h_{l,g}^\top, \quad \nabla_{A_{l,g}} \mathcal{L} = (J_{l,g}^\top B_{l,g}^\top \bar{y}_l) x_l^\top. \quad (42)$$

The output matrix sees the realized feature vector $h_{l,g}$ directly. The input matrix sees the upstream gradient filtered through $B_{l,g}$ and the rational group Jacobian. This is the a priori reason MatrixPolicy uses $o_{l,g}^{\text{live}} = \text{rms}(R(u_{l,g}))$ for the output role and $d_{l,g}^{\text{live}} = \text{rms}(R'(u_{l,g}))$ for the input role.

B.2 Relative pressure as a log-scale proxy

The exact derivative along the homogeneous rescaling direction would be

$$\Delta_g = \langle \nabla_{A_{l,g}} \mathcal{L}, A_{l,g} \rangle_F - \langle \nabla_{B_{l,g}} \mathcal{L}, B_{l,g} \rangle_F. \quad (43)$$

MatrixPolicy does not use this signed quantity directly. Instead, it uses RMS relative pressures because they bound the magnitude of a log-RMS change under a small gradient step. If $W^+ = W - \eta G$ and η is small, then

$$\log \text{rms}(W^+) - \log \text{rms}(W) = -\eta \frac{\langle G, W \rangle_F}{\|W\|_F^2} + O(\eta^2), \quad (44)$$

and Cauchy-Schwarz gives

$$\left| \frac{\langle G, W \rangle_F}{\|W\|_F^2} \right| \leq \frac{\|G\|_F}{\|W\|_F} = \frac{\text{rms}(G)}{\text{rms}(W)}. \quad (45)$$

Thus $p_{l,g}^{\text{in}}$ and $p_{l,g}^{\text{out}}$ are not natural-gradient terms; they are scale-normalized proxies for how aggressively each matrix role is being moved. The log difference $\pi_{l,g}$ in equation 9 compares these proxies in a way that is invariant to common multiplicative changes of the loss scale.

B.3 Centered group policy preserves the mean log step

Before clipping, the group policy is recentered by

$$\hat{c}_{l,g,r} = \frac{c_{l,g,r}}{\text{geomean}_j c_{l,j,r}}. \quad (46)$$

Therefore

$$\frac{1}{G} \sum_{g=1}^G \log \hat{c}_{l,g,r} = \frac{1}{G} \sum_{g=1}^G \log c_{l,g,r} - \log (\text{geomean}_j c_{l,j,r}) = 0. \quad (47)$$

This is why the group policy is a redistribution rule rather than a hidden global learning-rate schedule. Clipping can slightly perturb the exact zero-sum log property, but it prevents a noisy group statistic from dominating the matrix update.

B.4 Bounded rescaling is a contraction toward a target ratio

Let

$$\gamma_{l,g} = \log \text{rms}(A_{l,g}) - \log \text{rms}(B_{l,g}). \quad (48)$$

On an active positive-rescaling step, $A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}$ and $B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}$, so the log ratio changes as

$$\gamma_{l,g}^+ = \gamma_{l,g} + 2\ell_{l,g,t}. \quad (49)$$

Ignoring clipping for the moment and writing $\lambda_{l,g,t} = s_{l,t} a_{l,g}^{\text{rs}}$, the update in equation 28 gives

$$\gamma_{l,g}^+ = (1 - \lambda_{l,g,t})\gamma_{l,g} + \lambda_{l,g,t}\tau_{l,g}. \quad (50)$$

Thus each active rescaling step moves the matrix representative toward the target ratio without changing the represented homogeneous block map. In the implemented setting this operation runs every five optimizer steps, $\lambda_{l,g,t}$ is small, and the log move is clipped by 0.030, so it acts as a gradual balancing term rather than a large functional update.

B.5 Why the Muon branch is early

The branch fraction in equation 21 contains the window

$$w(\xi_t) = \text{smoothstep}(0.02, 0.12, \xi_t) [1 - \text{smoothstep}(0.20, 0.36, \xi_t)]. \quad (51)$$

It is approximately zero at initialization, rises during the early phase, and then decays to zero. This shape matches the role of the RLB matrices. Early in training, changing matrix directions changes the normalized coordinates seen by the rational curves; later, after the curves and surrounding Transformer layers co-adapt, a persistent Muon branch would add an additional optimizer mechanism where the simpler role/depth AdamW branch is sufficient. The activity penalty $\psi_{l,t}$ further reduces the branch when the rational coefficients are already moving strongly, coupling the branch to the state of the activation rather than applying a generic matrix rule everywhere.